

Swap the Ports, Keep the Code, Read the Ledger: Deterministic Simulation Testing for an LLM Agent Harness

Compliance note (binding, from SCOPE.md — reconciled against the D28 closing note, HEAD b3953a9, 2026-08-09): every unqualified use of “DST” for the generated axis must name the profile. Three profiles exist: `driver_only` v22 (installs nothing; extension-model coverage **zero**), `driver_plus_no_ops` v9 (**non-zero and entirely of no-ops** — 32 hooks / 4 extensions, 16 satisfying criterion 2 vacuously), `driver_plus_compose` v1 (compose installed — the first effectful extension; 7 hooks covered, `on_tool_handle` excluded). The honest coverage claim at HEAD is the computed vacuity register: **exactly one of forty classification entries rests on a measured AND substantive basis** (`compose/on_response_intercept`); hermeticity is enforced **per profile** (compose’s registration runs ambient, gap stated not closed); the `extension_effect_fault` class is waived by every profile. The label covers the generated axis only. §0, §6, and §8 each carry part of this load — the compliance pass checks all occurrences. **The spine doc (refreshed 2026-08-07) predates D27/D28 and is stale on all of the above — source coverage claims from the D27/D28 NOTES, not from it.**

Abstract

LLM agent harnesses fail at the boundaries between the agent loop and its nondeterministic surroundings — provider, tools, approvals, environment, clock — in multi-step patterns that neither unit tests nor live-model evaluations can serve as a regression oracle for. We describe the deterministic simulation testing (DST) system of Motoko, an autonomous coding agent harness, in which the simulated environment is a conversational model provider rather than a network or disk: production transition code runs unchanged against a seeded deterministic world and emits a typed event ledger checked by structural invariants, with seeded discovery generating the ordering of environment events, a logical-fault catalogue whose classes demonstrably reach named production recovery branches, a virtual clock observed by production timeout behavior, and strict replay of the exact recorded execution program. Distinctively, the project reserved the unqualified word “DST” behind a written seven-pillar conformance bar and adopted it only when an eleven-row automated acceptance gate passed — first for one deliberately empty execution profile (`driver_only`), then re-run across three profiles including one installing an effectful extension — and it computes rather than asserts what its green results rest on: at the current revision, exactly one of forty coverage-classification entries stands on a measured and substantive basis, a number the system prints and guards. We present the ports-swap architecture, the deterministic test world, and this conformance-and-vacuity accounting as a transplantable methodology for testing agent infrastructure.

1. Introduction

An LLM agent harness is the software around the model: the loop that assembles context, calls a provider, executes the tools the model requests, applies approval policy, compacts the conversation when it grows, and decides when to stop. Motoko is such a harness — an autonomous coding agent whose loop, extension hooks, compaction, checkpointing, and tool-approval protocol are the

production code under test in this report. The harness matters because it is where an agent system’s *reliability* lives: the model may be stochastic by design, but the machinery around it is ordinary software with ordinary bugs — and, as it turns out, with a characteristic bug shape.

That shape is the multi-step boundary failure. The failures that motivated this work were not wrong functions but wrong *interactions across steps*: provider telemetry from step N silently shaping step N+1; a conversation compaction applied to the payload sent to the provider but not to the loop’s own history; an extension hook shown the wrong slice of the conversation; a harness environment variable silently disabling a behavior the tests believed was on. Each of these is invisible to a unit test, because no single pure function is wrong, and each is invisible to a live-model evaluation, because a live run that behaves badly cannot be reproduced, minimized, or re-run after a fix with the same inputs. Evals measure capability; they cannot serve as a regression oracle for the harness itself. Live-provider runs survive in Motoko’s tree only as supplemental calibration smokes, explicitly outside the deterministic oracle.

The classical answer to this problem class — in databases, not agents — is deterministic simulation testing: make the entire environment a deterministic function of a seed, run the real system inside it, assert invariants over the resulting trace, and reproduce any failure from its seed. This report describes that idea transplanted to a domain where the environment to be simulated is not a network or a disk but a *conversational model provider*, plus tool execution, an approval channel, a process environment, and time. The transplant is scoped precisely: this is single-actor, **logical-fault** DST — physical fault models and multi-actor scheduling are excluded by recorded decision, with tripwires for the day the exclusions stop being valid, not by omission (§5.3, §8).

The report makes three contributions. First, an architecture — *swap the ports, keep the code, read the ledger* (§3, Fig. 1) — in which production transition code runs unchanged against scripted fakes and emits a typed event ledger that is the sole test oracle, layered from pure policy up to end-to-end deterministic sessions (§4). Second, a deterministic test world (§5): seeded discovery that generates the *ordering* of environment events by reacting to the requests production code actually makes, a logical-fault catalogue whose classes demonstrably reach named production recovery branches, a virtual clock that production timeout behavior observes, and strict replay of the exact recorded execution program. Third — and least common in the literature — a conformance-and-vacuity accounting discipline (§6): the project reserved the word “DST” behind a written bar, met it through an automated acceptance gate, and computes rather than asserts what its green results rest on, down to the honest statement that exactly one of forty coverage-classification entries at HEAD is both measured and substantive. §7 describes the gate and CI topology; §8 states the limits with the same precision as the capabilities.

2. Background and related work

Three primers carry the rest of the report: deterministic simulation practice, the AILANG effect system, and the Motoko loop itself.

2.1 Deterministic simulation testing

DST entered field practice through systems infrastructure. FoundationDB made simulation a first-class development mode: the database runs single-threaded inside a simulated network, disk, and clock, all deterministic functions of a seed, with faults injected by the scheduler and invariants

checked continuously. TigerBeetle’s VOPR applies the same shape to a state-machine replica set; Antithesis moves the determinism boundary down to a hypervisor so that unmodified software can be simulated. Beneath the differences, the shared contract is: a seed generates the environment’s event schedule; the *real* system code executes inside it; invariants judge the result; the seed (plus versions) is a complete reproduction key.

Neighboring methods each keep part of this contract. Property-based testing generates *inputs* from a seed but not the environment’s event schedule; trace replay reproduces an environment that was once observed but cannot exercise what was never recorded; golden-output testing couples assertions to surface output — fatal in this domain, where surface output is model prose. Motoko’s taxonomy decision (§5.1) orders these precisely, and the ordering matters to this report’s naming story: the system’s own seeded axis was classified as property-based testing over agent-loop state — “strictly stronger than trace replay, strictly weaker than DST” — until the full world existed.

For agent harnesses specifically, the testing literature is thin. Benchmark suites (SWE-bench-style) and harness-optimization work measure end-task capability of the model-plus-harness system; they are statistical instruments, not regression oracles, and they say nothing about whether a compaction preserved tool-call pairing. Regression testing of the harness itself, where discussed at all, is conventional unit and smoke testing. The gap this report addresses is exactly there.

2.2 AILANG effects and capabilities

Motoko is written in AILANG, and one language feature is load-bearing for the test architecture: algebraic effect rows with runtime capabilities. A function type declares the effects it may perform (`!{IO, Env, Clock, ...}`); the runtime grants capabilities per process (`ailang run --caps IO,Env`); *performing* an effect without its capability is a runtime failure at the perform site. Declarations are checked statically, capabilities dynamically, and the two are deliberately independent: a function may declare a maximal row and run under a minimal grant so long as it performs nothing outside the grant. §3.4 builds the suite’s conformance discipline on exactly this asymmetry.

2.3 The Motoko loop

The production system under test is a step loop. A pure decision function (`decide : StepState → StepDecision`) chooses the next action; model, tool, and hook phases execute against a ports boundary and return typed results plus state deltas; a driver (`session.ail`) threads the state, executes decisions, and emits every ledger event — it is the sole emitter, a property §3.3 depends on. Around the loop sits an extension system: third-party extensions bind hooks (`on_pre_step`, `on_tool_handle`, `on_response_intercept`, `on_solver_candidate`, ...) behind a versioned ABI, with a conformance kit released in lockstep majors that certifies compactor extensions against structural invariants. The ABI’s hook surface — which hooks are unconditionally dispatched, which effects each hook row declares — becomes central in §6, because it determines what an installed extension *could* do, and therefore what a profile that installs one has actually covered.

[FIG-1 appears in §3.]

3. Architecture: swap the ports, keep the code, read the ledger

Motoko’s testing architecture compresses to one sentence: *swap the ports, keep the code, read the ledger*. A production run and a deterministic run execute the same driver over the same transition

code; the only difference is which adapters sit behind the ports boundary, and the only oracle either run produces is the typed event ledger the driver emits. This section describes the five standing decisions and the three mechanisms that implement them.

3.1 Five decisions

The origin ADR fixed five decisions, and every later amendment — including the redefinition of what the word “DST” is entitled to mean (§5) — preserved all five:

1. **Model the external contracts deterministically.** Everything nondeterministic around the loop — the model provider, tool execution, the approval channel, the clock, the process environment — is represented by an explicit deterministic model rather than sampled live.
2. **Drive the real production transition code.** Scenarios execute the code that ships, never a reimplementations of it; a test double of the loop would certify the double.
3. **Record boundary observations into a normalized trace.** What crossed the boundary is data, not a mock’s call log.
4. **Assert reusable structural invariants over the trace — never final model prose.** Claims about a model’s wording couple a test to one provider and one sampling; claims about structure — tool-call pairing, checkpoint-chain validity, event ordering, count conservation — survive both.
5. **On failure, report scenario id, seed, and trace.** The triple is the reproduction key, and it is the suite’s public interface (§3.5).

3.2 Ports and fakes

All external interaction flows through a ports boundary. In production the driver is wired to live adapters (`live_ports`, a live model provider); in a deterministic run the same driver is wired to scripted fakes: `stub_step.ail` supplies scripted provider steps (`Scripted/ScriptedStep`, with `prose_step`, `tool_step`, ... constructors), `scripted_ports.ail` supplies a `ScriptedPortsState` covering model, approvals, and clock, and `ext_fixture.ail` supplies extension-hook fakes. The driver cannot tell which wiring it received; the scenario chooses.

Two properties make the swap load-bearing rather than cosmetic. First, the fakes are *pure*: handing the driver a scripted world adds no new effects, which is what enables the capability discipline of §3.4. Second, the boundary is *complete enough to carry the whole session* — and this was earned, not assumed. The original scripted-step mechanism was a success-only record; approvals, native tool execution, and session time all bypassed the ports boundary, which is why the full deterministic world of §5 required changing effect routing and the terminal-trace contract rather than extending an enum. The project’s taxonomy review said this explicitly, and it shaped the entire execution plan: reaching the conformance bar “must not be estimated as a small `ScriptedStep` diff.”

3.3 The ledger is the trace

The driver — `session.ail`, which executes the pure `decide : StepState → StepDecision` policy of `step_machine.ail` and the ports-only model/tool/hook phases — is the **sole emitter** of ledger events. The ledger is a typed vocabulary (`LedgerTrace/LedgerRecord` in `phase_vocab.ail`), projected to a versioned JSONL schema on the wire. Scenarios assert over the typed in-memory

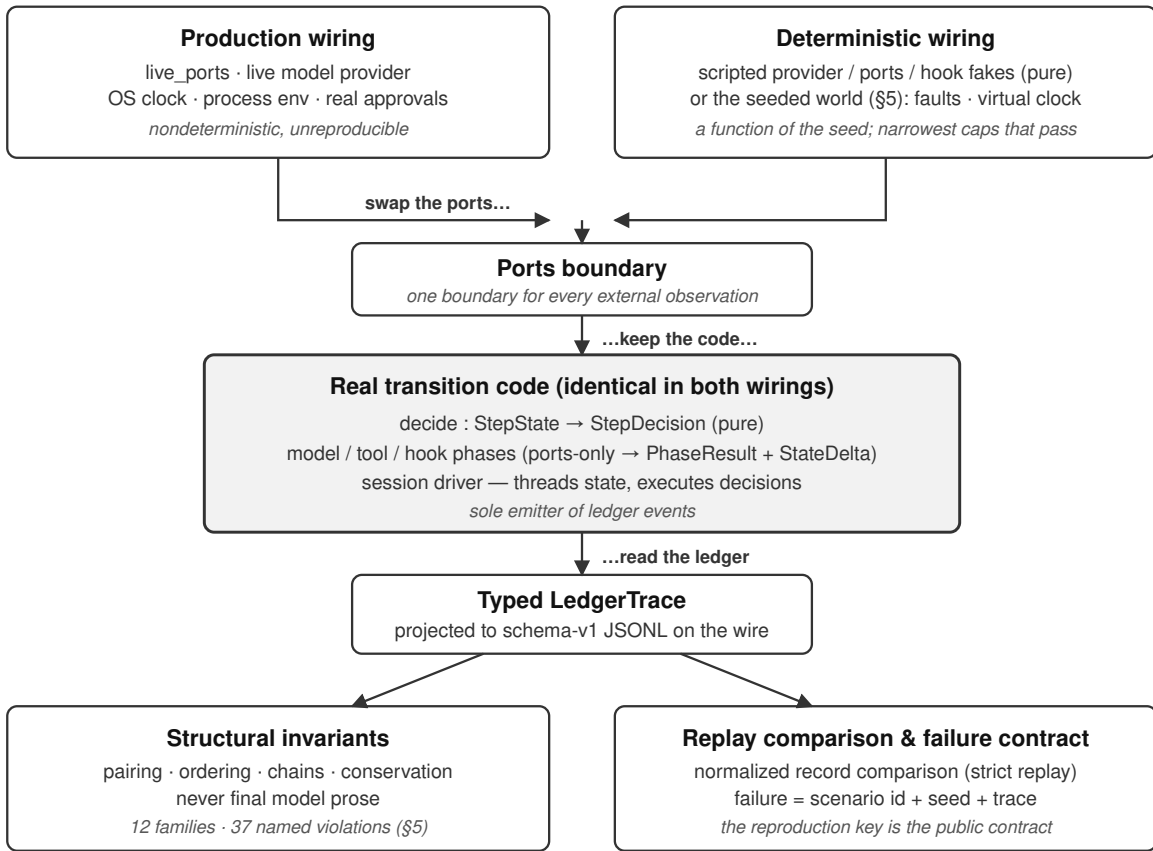


Figure 1: Swap the ports, keep the code, read the ledger: one driver, two wirings, one oracle.

ledger; replay comparison operates on normalized records (payload digests, stage records, extension-diagnostic events).

Sole emission is what makes “the ledger is the trace” a sound oracle claim: there is no second channel through which an interesting boundary observation could leave the system unrecorded, so an invariant over the ledger is an invariant over everything the run observably did. Where that claim has exceptions, the system enumerates them rather than hoping — §6 discusses the two vocabulary variants that do not reach the returned trace today, and the stated reading under which the completeness row passes.

3.4 Capabilities as a conformance check — and the stated limit

AILANG functions declare effect rows (! {IO, Env, Clock, ...}) and the runtime grants capabilities per process (ailang run --caps IO,Env); performing an effect without its capability fails at perform time. Because the fakes are pure, every deterministic gate runs under the **narrowest capability set that passes** — from --caps IO for pure-policy gates up to the full row for stubbed-AI gates. The capability flags thereby double as a conformance check: if a code path under test reaches for an effect *class* the world does not model, the run dies at the perform site instead of silently touching the host (“caps-as-conformance”).

The limit is stated as prominently as the mechanism, because the failure mode it leaves open is exactly the kind this system exists to catch: capabilities reject a new effect class, not a new unmodeled *operation within* an already-granted class. A granted IO still admits readLine(); a granted Env still admits a new variable read. Capabilities alone therefore do not prove hermeticity. Hermeticity is earned separately, by two-sided poison probes over the world boundary (§5.2), and — as §6.3 shows — it is earned *per profile*.

3.5 The failure contract, and the division of labor with proofs

Every failing scenario reports scenario=<id> seed=<seed> invariant=<failed_invariant> plus trace <line> lines through a shared harness. Scenario ids are dotted, layer-prefixed names (phase_c.11.*, compaction.gen.tool_heavy, ...) and are the suite’s stable public contract: gate output is parsed by humans and scripts, so renaming or dropping an id is treated as a wire-schema change — deliberate and reviewed. The script that houses a scenario is an implementation detail; the id is not.

The deterministic suite coexists with a small proof layer rather than competing with it. make_verify_core discharges Z3 contracts for local universal properties of pure helpers (advisory in CI); the deterministic suite owns anything that requires execution traces across time or process boundaries. The division pays in both directions: proved helpers keep scenario invariants from re-deriving policy arithmetic, and scenarios cover exactly the multi-step state that pointwise proofs cannot see.

4. The layered test stack

The deterministic suite is organized in four layers that share one scenario harness, one id namespace, and one anti-silent-drop discipline. A fifth axis — seeded parameter generation — cuts across the layers and is the historical predecessor of the full test world of §5.

4.1 Four layers

[TBL-1: the layer stack.]

Layer	What it tests	Runner
L0 — pure policy	thresholds, token estimates, elision, invariant predicates; Z3 contracts where provable	<code>ailang run/ailang test;</code> <code>make verify_core</code> (advisory)
L1 — loop state	the real <code>session/step_machine</code> against scripted ports: compaction chains, checkpoints, guards, the approval protocol	<code>make phase_c_l1</code> , <code>make compaction_dst</code>
L2 — harness boundary	the TypeScript harness <i>before</i> AILANG starts: system-prompt materialization, env/spawn preparation	<code>make dst_l2</code> (bun)
L3 — end-to-end deterministic	the real driver with fully scripted ports; event parity between phase paths by two-capture diff	<code>make smoke_parity</code>

Each layer answers a class from §1’s failure taxonomy. L0 keeps policy arithmetic honest where it is pure. L1 is where most scenarios live, because most harness bugs are loop-state bugs: a compaction applied to the send payload but not the history is invisible to any single function’s unit test and obvious to an invariant over a scripted three-step session. L2 exists because a harness can be broken before the agent process starts — the environment and system-prompt materialization are tested in the TypeScript layer that performs them. L3 closes the loop: the full driver, every port scripted, checked for event parity across code paths that must agree.

4.2 One harness, one exception

Scenarios share a single in-repo runner, `dst_harness.ail`:

```
export type ScenarioFailure = { failed_invariant: string, trace: [string] }

export type Scenario = {
  id: string,
  seed: string,
  run: () -> Result[(), ScenarioFailure]
  ! {AI, FS, Process, IO, Env, Net, SharedMem, Clock, Stream, Trace, Rand}
}
```

The `Scenario` type carries the *maximal* effect row, yet IO-only gate scripts still run under `--caps IO`: unused effect rows demand no capabilities — only performed effects do — so the narrowest-caps discipline of §3.4 survives type-level generality. One deliberate exception exists: the extension-conformance kit (§2.3) keeps a package-owned harness, because sharing the in-repo type caused a

transitive name-resolution collision with the ABI package and the kit’s surface is ABI-versioned — consolidating would have forced a major version for zero behavior change. The exception is policed, not tolerated silently: `rg '^func run_all' scripts` must stay empty.

4.3 The namespace, and counting as an oracle

[TBL-2: scenario-id namespace with per-gate counts — populate from gate `PASS count=N` output at HEAD, not from documentation. [VERIFY] Doc-derived snapshot: ~65 fixed scenarios across `compaction.*`, `runtime_status.*`, `phase_c.l1.*` (15), `phase_c.approval.*` (7), `phase_c.c2.*` (18), `conformance.compactor.*` (4, × hook cases), `harness.*` (7), plus five seeded `.gen.` families and the RNG canary.]

The suite defends its own completeness with a counting discipline. Every gate prints a pass count *derived* from the summed per-scenario results, never a literal; removing a scenario moves the count, and a dangling reference fails to compile. Adding a scenario must move the owning gate’s count by exactly the number added. The same rule extends to the seeded axis (`families=N` is computed from the family list) and, at the top, to the umbrella gate’s target list, which is expanded by `make` itself so a target cannot silently fall out of the sweep. The principle throughout: the suite’s inventory is itself an asserted output, not a maintained document.

4.4 The seeded axis: parameter generation, honestly classified

Before the full test world existed, the suite grew a seeded axis: five generated families (four over the phase-C loop — seal boundary, checkpoint pressure, system-prefix split, stage-record projection — and one tool-heavy compaction family), run at `DST_SEEDS` seeds from `DST_BASE_SEED`. A failure reproduces with `DST_BASE_SEED=<seed> DST_SEEDS=1 make dst_seeded`.

Two property rules govern every family, both learned from failure modes rather than taste. First, **invariants only**: a generated family that asserts a specific decision will flake or go vacuous, because legitimate draws produce legitimate variety (short-content draws correctly yield `PassThrough`); permitted decision assertions must be one-sided — *below threshold* → *never compacted*, with no converse. Second, **import policy constants, never hardcode tiers**: a family with a duplicated threshold keeps certifying the old policy forever after a config change.

The axis carries one fixed sentinel, the **RNG canary**: `rand_seed(12345)` pinned to golden integers. If the language’s PRNG ever changes, the canary fails loudly instead of every recorded seed in every past bug report silently re-mapping to a different case. It is the reason a seed is a durable reproduction key at all.

The project’s own classification of this axis is worth reproducing because it motivates §5: seeds here draw *input parameters*, never the ordering of environment events; there is no fault injection and no clock. The taxonomy review scored it accordingly — “strictly stronger than trace replay, strictly weaker than DST”; in its scorecard’s words, property-based testing over agent-loop state, not simulation. That verdict, and the refusal to use the stronger word until it was earned, is where the next section begins.

5. The deterministic test world

Everything described to this point, the project refused to call DST. In July 2026 a taxonomy review froze that refusal into a written bar; over the following weeks the project built the integrated capability that meets it. This section describes the bar, then the world.

5.1 The bar

The taxonomy decision reserves the unqualified words “DST” and “simulation” — in code, CI target names, and documentation — for a seven-pillar conformance profile at the single-actor, logical-fault level: (1) hermetic determinism; (2, logical) one deterministic model of provider, tool, approval, extension, and time outcomes, including a fault catalogue; (3) the seed generates the *trajectory of environment events*, not merely input values; (4) the seed injects logical faults from a normative minimum catalogue; (5) a virtual clock advanced by the world and observed by production time-bearing behavior, with timeouts reachable; (6) invariants over the returned `LedgerTrace`; (7) a reproduction contract of seed plus generator and schema versions plus the exact execution program. Physical faults and concurrent scheduling are explicitly out of profile. All required pillars must hold *together*: until 3, 4, and 5 are met, the method is not DST regardless of how much other machinery exists.

Two features distinguish this from a definition adopted by usage. First, at the moment it was written, the system did not meet it — the same document scored HEAD as “PBT, not DST” and assigned the honest interim name. Second, the bar is enforceable only with automated evidence, and the decision enumerates the six criteria a reviewer applies to behavior, not to the presence of types: a recorded seed must produce a typed program of multiple ordered external instructions whose kinds and order were generator choices, *consumed by the real session driver as responses to requests production code issued*, with a later instruction’s kind able to depend on an earlier production request; required fault classes must reach named production recovery branches; a pair of programs differing in virtual-clock advancement must produce different timeout behavior; the traced driver must return exactly one terminal record for every enumerated terminal path, with invariants over the returned trace; the exact serialized program must replay without invoking the generator; and a hermeticity probe must show that execution-relevant effects cannot silently bypass the controlled world. The criteria were written with the failure modes of pretend-simulation in mind — the first explicitly rules out a generator-chosen list consumed by a pure projection, which one existing seeded family did.

5.2 The world, as built

The architecture that meets the bar is a driver-owned, state-threaded execution-world protocol. Conceptually, one boundary function:

```
handle(world_state, world_request)
  -> Result({ intermediate_emissions, response,
            next_world_state, observations }, HarnessError)
```

The real session driver owns `world_state` and threads it beside its loop state; every external observation that can affect control flow or the ledger — provider steps, typed tool outcomes, approvals, environment reads, runtime randomness, conformant extension effects, logical resource state, and time — crosses this one boundary. Nothing may hide a cursor or clock in shared memory, process globals, or an ambient RNG; the explicit value in `world_state` is the only clock. (The prohibition

was validated the hard way: a variant that placed the provider cursor inside the world adapter type-checked, kept every gate green, and silently replayed step 0 forever — exactly the class of wrong-but-green outcome the discipline exists to exclude.)

Discovery and replay are distinct modes. Seeded *discovery* reacts to the external requests production code actually makes: the generator chooses modeled outcomes, logical faults, and virtual latencies as the run unfolds, and records the resulting ordered, versioned **ExecutionProgram**. *Strict replay* consumes that exact serialized program without invoking the generator and compares normalized terminal traces. The program, not the seed alone, is the reproduction key — a seed re-derives the program only while the generator version stands still, and the program survives generator evolution. Recorded programs carry their execution manifest inside the artifact.

The remaining pillars, concretely:

- **Faults are modeled outcomes, not injected internal decisions.** The catalogue’s nine required classes span provider errors (retryable, non-retryable, protocol-inconsistent, partial-stream-then-error, empty-terminal), typed tool failures (**ToolFailed**, **ToolCorrelationMismatch**, **ToolDeadlineExceeded**), and approval denial. All nine reach named production recovery branches (Table 3) — eight found **by search** over seeds, one by construction, with the constructed case carrying its own measured justification (0 of 260 swept seeds reach it).

Table 3 — the fault catalogue: class → named recovery branch. The class-to-branch map is printed in full by the gate on every run, so a class that quietly lost its branch is visible in CI output. (Source: `src/core/dst_fault_catalogue.ail`; reached/waived status per the D28 rerun.)

Fault class	Recovery branch	Status
<code>provider_error_retryable</code>	<code>session.c2_loop/stream_error_retry</code>	reached by search
<code>provider_error_non_retryable</code>	<code>session.c2_loop/provider_failure_finalize</code>	reached by search
<code>provider_protocol_inconsistent</code>	<code>tool_dispatcher_adapter.../malformed_arguments</code>	reached by search
<code>provider_partial_stream_then_error</code>	<code>session.c2_loop/stream_error_retry</code>	reached by search
<code>provider_empty_terminal_response</code>	<code>session.c2_loop/empty_stop_finalize</code>	reached by construction
<code>ToolFailed</code>	<code>tool_phase.tool_outcome_message/ToolFailed</code>	reached by search
<code>ToolCorrelationMismatch</code>	<code>tool_phase.tool_outcome_message/ToolCorrelationMismatch</code>	reached by search
<code>ToolDeadlineExceeded</code>	<code>tool_phase.tool_outcome_message/ToolDeadlineExceeded</code>	reached by search
<code>approval_denied</code>	<code>session.c2_loop/approval_denied</code>	reached by search
<code>approval_deadline_exceeded</code>	<code>tool_phase.resolve_approval/waived</code>	waived (production-policy condition)
<code>extension_effect_fault</code>	<code>compaction_ai.compact_with_availability</code>	reached by search (\$6.3)

- **Virtual time matters, demonstrably.** A latency pair with identical provider script, approval queue, and declared deadline — differing only in clock advancement — makes the same tool request, yet 4 of 9 native tool results in the slow half (and only the slow half) carry **ToolDeadlineExceeded**, with no OS timeout involved. Every profile-reachable time-bearing read in core routes through the world clock; the routed-set claim is *computed at the bound revision* (7 reachable sites, 6 routed, 1 declared and instrumented — the live adapter).

- **Hermeticity is probed two-sidedly.** Five poison pairs each show the deterministic world *completing* with a capability withheld (AI, Clock, Env, FS, Process) while the live wiring *dies* under the same withholding — evidence that the deterministic path genuinely does not touch the host, and that the probe itself is capable of detecting it if it did. A one-sided probe would prove only the former.
- **Every run returns one complete terminal trace,** through a single finalization path enforced structurally, and whole-execution invariants — twelve families, thirty-seven named violation constructors — evaluate over the returned trace.

5.3 What the world is not

The world is sequential and single-actor by decision, not omission: no physical disk or network transport faults, no concurrent scheduling, with revisit tripwires recorded for the day the production contract changes. The simulator models the environment an agent harness actually has — one driver, one provider conversation, logical failures at typed boundaries — and the project declined the prestige of the harder profile it does not need. §8 returns to these limits alongside the coverage limits of §6.

6. Earning the name: conformance and vacuity accounting

The previous section described machinery. This one describes something rarer: the bookkeeping that says what the machinery’s green results actually rest on. The naming gate is itself a test artifact — an eleven-row acceptance table run against automated evidence — and its most instructive feature is not that it passed but how precisely the record states *which parts of the pass were vacuous*, and what has happened to each vacuity since.

6.1 Reserving a word behind a bar

On 2026-07-24 the taxonomy decision ruled that the system was not DST and should not be called DST until the bar of §5.1 was met. The ruling had teeth in both directions: every existing **dst** identifier — target names, script names, this very vocabulary — was explicitly grandfathered as historical usage, and every *claim* was renamed to the honest interim label. Thirteen days later, on 2026-08-06, the acceptance gate (WI-D5) ran the eleven-row table and found eleven passes, and the unqualified label was adopted — with a scope statement attached to the adoption itself: the label covers the **generated axis only**, under **one named profile, driver_only/10**, whose extension-model coverage is zero. Four of the eleven rows passed by leaning on that emptiness, and the gate printed the accounting rather than leaving it to a reviewer to notice. Three days later the closing rerun (WI-D28, at v22 of the same profile) re-ran all eleven rows over three profiles’ worth of evidence and reached the same verdict with a different character — in the record’s own words, the name “now rests on a demonstration rather than on an absence of counterexamples.”

One row deserves calling out before any table, because it is the honesty mechanism working in public. Row 7 (oracle completeness) passes — at the gate and again at the rerun — **on a stated reading**: “logical ledger emissions” means emissions the axis can produce, not every logical variant in the vocabulary; two variants are enumerated as not reaching the returned trace today. The record states, twice, that if a reviewer rejects this reading the row is red and the verdict is NO. A

conformance claim that names its own single interpretive load-bearing point is worth more than one that does not, and this is the pattern this section is about.

6.2 The acceptance table, run twice

[TBL-A / App. A: the eleven rows — question, D5 verdict, D28 verdict, delta. Source: D5 note §“The eleven answers”; D28 note §1.]

The two runs bracket the interesting period, and the deltas tell the story better than either snapshot. Row 3 (“is the tested boundary honest?”) passed at D5 *vacuously* in every installed-extension clause, because the profile installed nothing; at D28 it is “the row that changed most” — the coverage floor, the exclusion rule, and the per-extension disclosure are all exercised non-vacuously across the two newer profiles, and the one clause still vacuous in all three is precisely identified (and belongs to a *producer*, not a profile — a distinct finding). Row 5 (“does virtual time matter?”) was real at D5 but explicitly non-transferable; at D28 it is **re-earned on routing**: the first profile whose installed extensions read a clock at all does so through the world’s routed port, with zero ambient clock sources measured in any hook path. Rows 6 and 9 now hold what the record calls their strongest instances in the tree — a real traced session with an installed effectful extension, recorded, validated, and strictly replayed to an identical interaction log. Row 10 changed shape rather than strength: hermeticity became a *per-profile* statement (§6.3).

The through-line is the register the closing note computes: **all four vacuities the gate reported at D5 still exist at D28; three had their ground replaced by a measurement; none was removed.** A lean is retired here only by measuring the thing it leaned on — never by re-wording the claim.

6.3 Profiles as the unit of coverage accounting

A profile is a named, versioned execution manifest, and the accounting rule — decided at adoption, before there was a second profile to tempt anyone — is that **coverage is earned per profile and inherited by none.** Three profiles exist at HEAD:

- **driver_only v22** — installs nothing. Extension-model coverage **zero**: the baseline the name was earned on, kept deliberately empty. It emits no coverage **STATEMENT** line at all — having nothing to state is itself stated.
- **driver_plus_no_ops v9** — four extensions, 32 covered hooks, and a computed sentence attached to the number: coverage is “NON-ZERO and ENTIRELY OF NO-OPS ... of which 16 satisfy criterion 2’s port and origin-tag clauses VACUOUSLY, i.e. over an empty set of performed effects. ZERO covered hooks mediate the world substantively.” A guard tool fails any run that states the 32 without the qualifier.
- **driver_plus_compose v1** — the first profile whose install set performs effects: seven hooks covered and one **excluded** (`on_tool_handle`, the ABI’s one gated slot, re-derived from the dispatch table rather than asserted), the first non-empty exclusion list in the tree. Its statement: seven hooks covered, “of which 1 mediate the world SUBSTANTIVELY through a D1 port and 2 satisfy criterion 2 vacuously.”

The tree-wide summary is a fold over classification lines the profiles already print, and it is the most important number in this report. Forty classification entries exist across the three profiles. Nineteen are world-mediation entries, of which eighteen are satisfied vacuously. Twenty-one of the

forty rest on an assumed rather than measured basis. **Exactly one of forty entries rests on a basis that is both measured and substantive:** compose’s `on_response_intercept`, witnessed by discovery under `driver_plus_compose` v1 — and the witness is existential, a record of what that run performed, not a bound on what the extension could do. The register’s own framing is the right one: the goal asked for *a* demonstration and this is it; “stating it as one-of-forty is the difference between reporting the demonstration and overclaiming from it.”

Two entries in that register show the discipline under load. First, hermeticity: the five poison pairs of §5.2 are `driver_only`’s discipline, and they do not extend across `driver_plus_compose`’s registration, which reads environment and files ambiently before any hook is dispatched (capabilities are per-process). The profile’s own output states the gap and names what carries the determinism claim instead — record-to-strict-replay identity, shown load-bearing by a mutant that turns the row red when the subject is removed. Second, the one fault class no profile exercises: `extension_effect_fault` is waived by all three profiles, on three different grounds — by construction (nothing installed), by measured inapplicability (zero port calls in any installed closure), and by a per-field fact (zero `ai_step` calls; compose’s only provider path sits behind its excluded slot). The record prices the change that would exercise it (a profile installing `compaction_ai`) rather than declaring the class covered by adjacency.

The accounting even tracks *why* an emptiness holds. The gate-era record believed `driver_only`’s empty install list was **forced** by an ABI barrier; a later work item measured all fifteen bindings of the barrier hook with two independent producers and narrowed the ABI, making the emptiness **chosen**. The record classifies this correctly as a *weaker* statement — a chosen emptiness covers exactly as much as a forced one — and the leaning rows’ reasons “concentrated rather than closed.” Separately, a goal-line clause requires every one of the fifteen extensions in the tree to be **mediated or disclosed with a measured reason**, and the closing note’s per-extension table delivers it, printing both analysis units wherever they disagree rather than promoting one silently.

6.4 Methodological sidebar: superseded claims stay on the page

A convention runs through every document cited in this section: a claim superseded by later work is kept verbatim, dated, and annotated — “kept because it was true when written” — rather than edited in place, because re-dating it would make it a false claim about history. The framework’s own reference doc still carries the sentence “the framework has no test scheduler, no injected-fault mechanism, and no virtual clock,” wrapped in a dated block recording exactly when each clause expired. The report’s authors recognize this as the same discipline the trace system applies to execution: documentation as an append-only ledger, where corrections are new records rather than rewrites of old ones. For a codebase whose tests exist to prevent silent revision of behavior, preventing silent revision of *claims about behavior* is the consistent move — and it is what made §6.2’s two-snapshot comparison possible to write at all.

7. Gates, CI, and operations

One umbrella target runs everything deterministic. `make dst` sweeps forty-nine gate targets (per `make dst_target_list` on the parallelization branch; [VERIFY] at land — the list is the source, not this sentence), from the pure-policy and loop-state gates of §4 through the world-axis gates of §5 (discovery, strict replay, execution program, fault catalogue, world state, latency pair, the three profile gates) to inventory self-tests and the L2 bun suite. The sweep has a deliberate two-phase

structure. One target, the PR fault corpus, is *wall-clock-gated* — its pass condition includes its own elapsed time, because the corpus’s declared seed minimums are arithmetic over its measured per-seed cost, and a time measurement taken on a loaded machine is a fact about the machine rather than the corpus. It therefore runs alone, first, under the exact conditions its ceiling was measured under; everything else then fans out in parallel (`-j$(DST_JOBS)`, per-target compile-cache lanes to keep concurrent compilation from interleaving, `--output-sync` per target). The exit code is the worse of the two phases: a red target in the fan-out cannot hide behind a green timed phase, or vice versa. A closing summary script reports per-target status against a *named* known-red list — a list that cannot outlive its failures, because the summary also reports any listed target that passes (Figure 2).

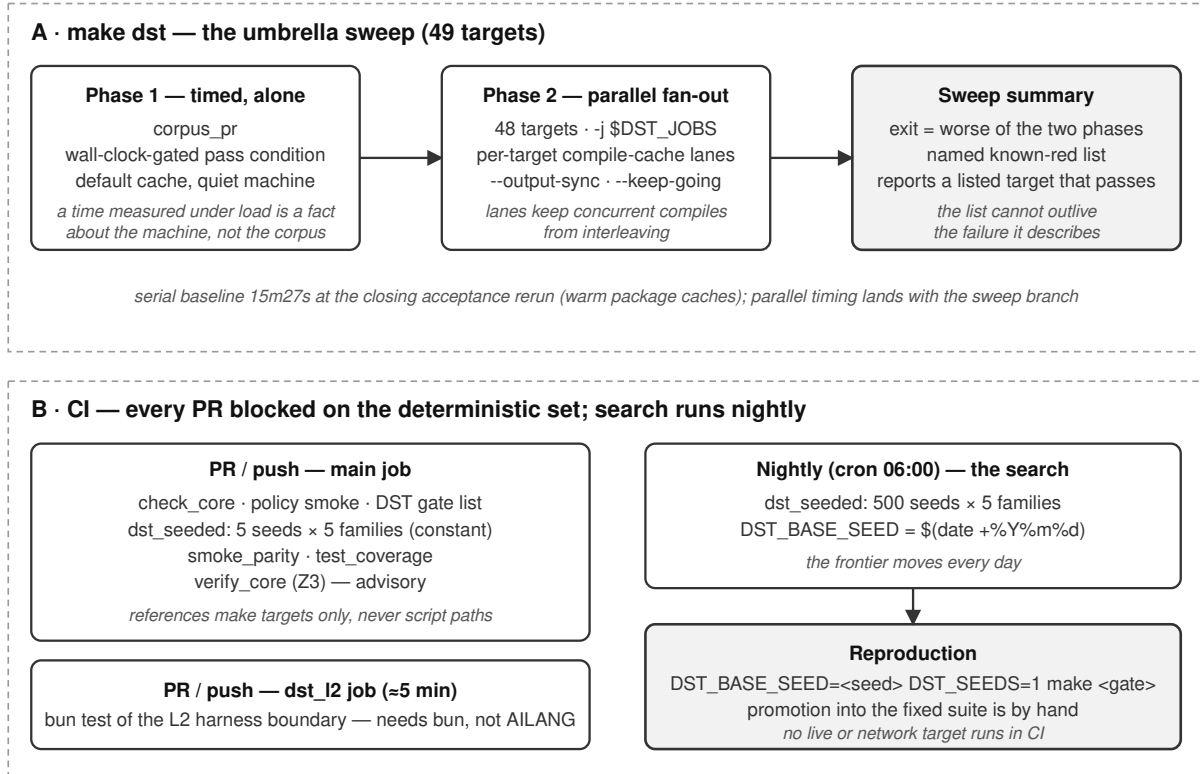


Figure 2: The umbrella sweep and CI topology: constant-time PR gates, nightly search.

Continuous integration blocks every pull request on the deterministic set and nothing else. The main job runs the core type-check, a policy smoke, the DST gate list, event parity, and the inline-test coverage sweep, with the Z3 contracts advisory; a separate five-minute job runs the L2 harness-boundary suite (it needs bun, not AILANG). Two standing rules keep the wiring stable: CI references make targets only, never script paths — file moves are absorbed by the Makefile, which is what let an entire script-directory migration land without touching the workflow — and no live or network target runs in CI; live-provider calibration stays manual.

Search is scheduled where it is cheap and bounded where it is not. On pull requests the seeded axis runs a fixed, constant-time configuration: five seeds per family from base seed 1. On the nightly schedule it runs five hundred seeds per family from a *date-derived* base (`DST_BASE_SEED=$(date +%Y%m%d)`), so the search frontier moves every day and any nightly failure reproduces locally from

that day's date. The world axis searches by corpus instead: the PR bank holds a declared minimum of seeds (twelve, with fifteen banked and thirteen affordable within the five-second budget at the measured 381 ms/seed) across three member kinds — fixed, promoted-regression, constructed-for-class — while a rotating gate walks six contiguous 240-seed epochs whose shards partition the window. Failing seeds are promoted into the fixed suite by hand; there is no automatic promotion (§8).

The reproduction contract, operationally: a seeded-axis failure line names its family and seed, and `DST_BASE_SEED=<seed> DST_SEEDS=1 make <gate>` replays exactly that case; a world-axis failure names its recorded program, and strict replay re-executes the program without the generator. The serial sweep's baseline cost is **15m27s** for the full `make dst` at the closing acceptance rerun (warm package caches; the record notes its cache methodology explicitly so the number is not compared against cold-cache runs). [VERIFY + MEASURE: parallel `DST_JOBS=n` wall-clock from the sweep summary once the parallelization branch lands.]

8. Discussion and limitations

The system's own discipline obliges this section to be as precise as the capability claims, so the limits below are stated in the record's terms, not softened summaries.

Scope is bounded by decision, with tripwires. The world is single-actor and logical-fault only. Both exclusions were *verified* at adoption rather than assumed — tool entries dispatch sequentially in list order, native batches are sequential, delegated calls are awaited one at a time; and no `fsync/WAL/recovery` path exists whose durability could be simulated — and both carry recorded tripwires: the single-actor exclusion must be revisited if parallel tool dispatch or ledger-sharing subagents land, the physical-durability exclusion if a persistence path appears.

Capabilities do not prove hermeticity, and hermeticity is per profile. §3.4's limit stands: a granted effect class admits new operations within it. The poison pairs prove the deterministic world completes without touching `AI`, `Clock`, `Env`, `FS`, or `Process` — for what they poison. Extension *registration* under the compose-bearing profile sits outside that discipline (capabilities are per-process, and registration reads configuration ambiently before any hook is dispatched); there the determinism claim rides record-to-strict-replay identity — a documented substitution, not an oversight.

Coverage is thin, and the system says exactly how thin. One of forty classification entries is measured-and-substantive; twenty-one rest on an assumed basis whose evidentiary amendment is drafted but unlanded; `extension_effect_fault` is waived by every profile; one acceptance clause is vacuous in all three profiles for a reason belonging to a *producer* rather than a profile — a vacuity class invisible to per-profile questions, found by asking each surviving exemption why it survives; and no run in the tree reaches an excluded dispatch (that mechanism is exercised by fixture only). Coverage is per-profile and non-transferable by rule. None of this is concealed by the green table, which is the point of §6.

Search is breadth without minimization. There is no shrinking, and no automatic promotion of a failing nightly seed into the fixed suite — promotion is by hand. The PR corpus's cost margin fits by exactly one seed (thirteen affordable against a declared minimum of twelve), and the record pre-commits to the honest response when the margin next tightens: raise the budget or lower the minimum, not re-measure until convenient.

The oracle has one stated interpretive choice. Row 7 passes on a reading — logical emissions that *occur* reach the trace — that the record flags as load-bearing in every document that touches it. Two vocabulary variants do not reach the returned trace today.

A note on authorship. Motoko’s codebase is AI-authored under a no-human-written-code constraint. We suggest this is why the methodology looks the way it does: derived-count oracles, capability conformance, an RNG canary, computed vacuity statements, and guard tools that fail runs for *stating a number without its qualifier* are all defenses against an author — human or model — that optimizes for green over true. The techniques transfer to human teams unchanged; the forcing function here was autonomy, where “the tests pass” is only meaningful if the tests police their own meaning.

9. Future work and conclusion

The near-term work is what the record itself prices. A profile installing the AI-compaction extension is the one change that would exercise the universally-waived `extension_effect_fault` class. Landing the drafted criterion-2 evidentiary-basis amendment would move twenty assumed-basis classification entries onto measured ground. The closure-versus-hooks analysis-unit promotion decision is open, with both units reported wherever they disagree. An executed excluded-dispatch would move that mechanism’s evidence beyond fixtures. Further out: seed shrinking and automatic promotion of nightly failures; the deferred legacy-smoke subsumption audit; in-flight extension diagnostics; and trace visualization tooling (a sibling project) for reading recorded programs and ledgers.

The report’s summary claim is methodological as much as architectural. The architecture — swap the ports, keep the code, read the ledger — carries deterministic simulation into the agent-harness domain, and the world of §5 shows that the full contract of the classical systems — seeded event orderings, logical faults, virtual time, strict replay — is achievable there. But the part we would most urge other harness builders to transplant is the discipline of §6: the project treated the word “DST” as a falsifiable claim with an acceptance test, adopted it only when the test passed, published which rows passed vacuously, and re-earned the vacuous ones by measurement — keeping every superseded claim on the page, dated, because it was true when written. Tests tell you what your system does; this bookkeeping tells you what your tests prove. An autonomous codebase cannot be trusted without the second — and, we suspect, neither can anyone else’s.

Appendix A — The acceptance table, both runs

The eleven rows of the naming gate, with the adoption-gate verdict (WI-D5, 2026-08-06, profile `driver_only/10`) and the closing-rerun verdict (WI-D28, 2026-08-09, HEAD `b3953a9`, three profiles). Wording condensed from the two notes; the notes are the record.

#	Row (the question asked)	D5 verdict (adoption)	D28 verdict (rerun) and delta
1	Does one seed generate an <i>execution</i> , not only values?	PASS	PASS — one control digest moved with the generator; the equal-count/different-digest property is unchanged
2	Is there a modeled logical environment?	PASS	PASS — the world grew a modeled filesystem (three-valued path class); answer unchanged
3	Is the tested boundary honest?	PASS — vacuous in every installed-extension clause	PASS — transformed: floor, exclusion rule, and per-extension disclosure exercised non-vacuously across two newer profiles; clause 3 still vacuous in all three (a producer vacuity)
4	Do injected faults reach production recovery code?	PASS — the extension-effect-fault waiver bought by the empty install list	PASS — identical numbers; the waiver held in all three profiles on three different grounds (construction / measured inapplicability / per-field fact)
5	Does virtual time matter?	PASS — real, and non-transferable	PASS — re-earned on routing: the first install set that reads a clock does so through the world's routed port; 0 ambient clock sources measured in any hook path
6	Is production logic under test?	PASS	PASS — strongest instance in the tree: a real traced session with an installed effectful extension

#	Row (the question asked)	D5 verdict (adoption)	D28 verdict (rerun) and delta
7	Is the oracle complete?	PASS on a stated reading ; ScratchpadResult exemption bought by emptiness	PASS on the same stated reading; the exemption re-earned twice on grounds that are not emptiness; two variants still do not reach the returned trace
8	Are harness failures separate?	PASS	PASS — the exclusion arm now has a profile with a non-empty exclusion behind it
9	Are discovery and replay stable?	PASS	PASS — strongest instance: a recorded, validated, strictly-replayed session with an effectful extension; censuses identical member for member
10	Is hermeticity enforced?	PASS	PASS — per profile : five two-sided poison pairs (<code>driver_only</code> 's discipline); <code>compose</code> 's ambient registration disclosed, determinism carried by replay identity
11	Is there actual search?	PASS	PASS — identical; cost margin unchanged and still thin (13 affordable vs minimum 12)

Appendix B — The shared scenario harness, in full

`src/core/test/dst_harness.ail` (64 lines) is the entire in-repo runner; it is reproduced nearly whole because its size is the point — the machinery of §§3–5 lives in the world and the gates, not in the harness.

```
export type ScenarioFailure = {
  failed_invariant: string,
  trace: [string]
}
```

```

export type Scenario = {
  id: string,
  seed: string,
  run: () -> Result[(), ScenarioFailure]
  ! {AI, FS, Process, IO, Env, Net, SharedMem, Clock, Stream, Trace, Rand}
}

export pure func failure(failed_invariant: string, trace: [string]) -> ScenarioFailure {
  { failed_invariant: failed_invariant, trace: trace }
}

export pure func ok_or_failure(ok: bool, invariant_name: string, trace: [string])
  -> Result[(), ScenarioFailure] {
  if ok then Ok(()) else Err(failure(invariant_name, trace))
}

export func report_failure(scenario: string, seed: string,
  f: { failed_invariant: string, trace: [string] }) -> () ! {IO} {
  let _ = println("scenario=${scenario} seed=${seed} invariant=${f.failed_invariant}");
  print_trace(f.trace) -- prints "trace <line>" per entry
}

export func run_one(s: Scenario) -> bool
  ! {AI, FS, Process, IO, Env, Net, SharedMem, Clock, Stream, Trace} {
  match s.run() {
    Ok(_) => { println("scenario=${s.id} ok"); true },
    Err(f) => { let _ = report_failure(s.id, s.seed, f); false }
  }
}

export func run_all(scenarios: [Scenario], failed: int) -> int
  ! {AI, FS, Process, IO, Env, Net, SharedMem, Clock, Stream, Trace} {
  match scenarios {
    [] => failed,
    s :: rest => {
      let ok = run_one(s);
      run_all(rest, if ok then failed else failed + 1)
    }
  }
}

```

Figure & table inventory

ID	What	Source	Status
FIG-1	Ports-swap: one driver, two wirings, ledger out	fig1-ports-swap.svg (this dir)	done (§3.2)

ID	What	Source	Status
FIG-2	Sweep structure + CI topology, PR vs nightly	<code>fig2-sweep-ci.svg</code> (this dir)	done (§7)
TBL-1	Layer stack L0–L3	spine doc, condensed	done (§4.1)
TBL-2	Scenario namespace + counts	gate <code>PASS count=N</code> output [VERIFY]	placeholder (§4.3)
TBL-3	Fault catalogue: class → branch → status	<code>dst_fault_catalogue.adb</code> + D28	done (§5.2)
App. A	Acceptance table, both runs	D5 + D28 notes	done
App. B	Harness listing	<code>dst_harness.ail</code>	done

Global [VERIFY] register (checked in the compliance pass)

- ~~Profile versions~~ **RESOLVED (reconciliation, 2026-08-10):** `driver_only` v22, `driver_plus_no_ops` v9, `driver_plus_compose` v1, at HEAD `b3953a9` per the D28 note. Re-check only if HEAD moves past `b3953a9` before the draft freezes.
- ~~Compose bearing profile~~ **RESOLVED: LANDED** (WI-D27, 2026-08-09; final rerun WI-D28). §6, §8, §9 and the compliance note updated in this pass. **The spine doc is stale on this** (refreshed 2026-08-07) — for coverage claims cite the D27/D28 NOTES; expect a spine-doc refresh to land and re-point citations then.
- All TBL-2 counts from gate output at HEAD.
- Seeded-generator control digests moved D5→D28 (`2144863192`→`722021275` vs `1372950750`, `n=23`; property unchanged) — quote D28’s numbers, and note a digest that moves when the generator changes is expected behavior.
- `mot-91` parallel sweep timings (§7) — measure post-land; serial baseline 15m27s recorded.
- Tripwire list quote (§8) from 007/ADR-001.
- Every “DST” occurrence names a profile; the compliance-note caveats (three-profile form) present (§0, §6, §8).
- Fault-corpus numbers at D28: 9 classes validated, 9/11 named recovery branches (2 waived with distinct reasons), wire witness counts reproduced exactly from D5 — cite D28 §row 4.